

NAME: AKSHAYA A

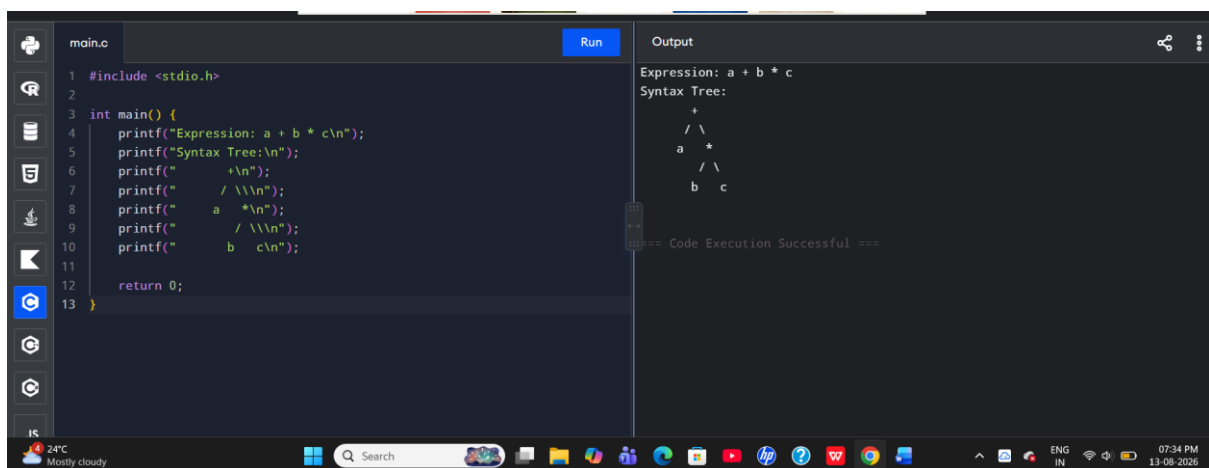
REG NO: 192421222

COURSE: CSA1402 COMPILER DESIGN

CO3_AT2_CASE STUDY

SET 1

Q1 – Parse Tree / Syntax Tree Representation



```
1 #include <stdio.h>
2
3 int main() {
4     printf("Expression: a + b * c\n");
5     printf("Syntax Tree:\n");
6     printf("    +\n");
7     printf("  / \
8     a  *
9     / \
10    b  c
11
12     return 0;
13 }
```

Output

Expression: a + b * c
Syntax Tree:

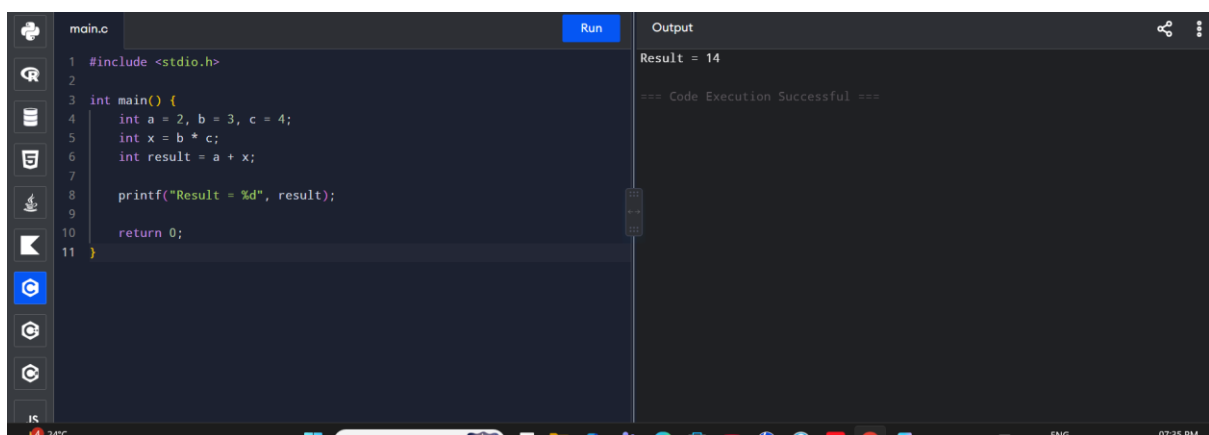
```

+
 / \
a  *
 / \
b  c

```

=== Code Execution Successful ===

Q2 – Bottom-Up Evaluation



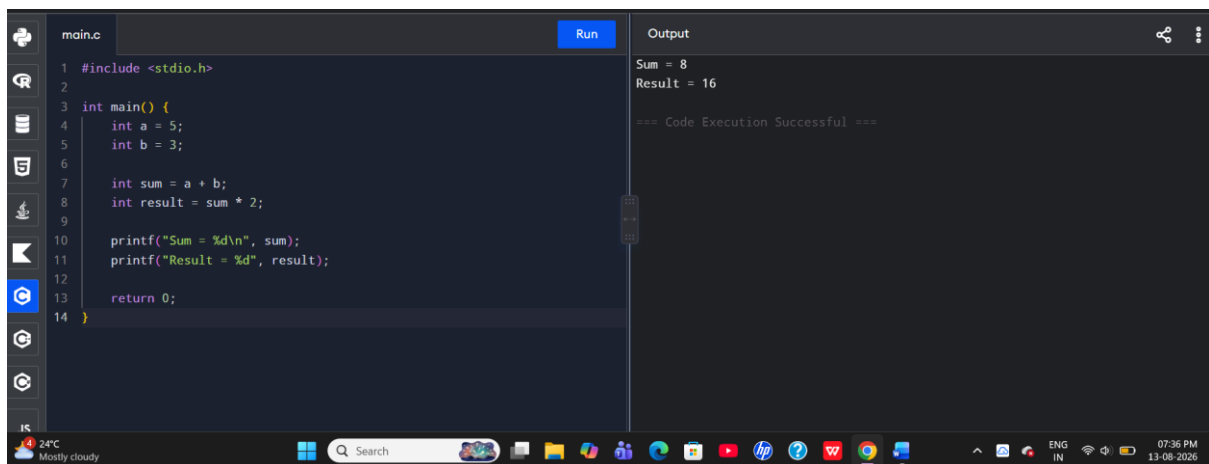
```
1 #include <stdio.h>
2
3 int main() {
4     int a = 2, b = 3, c = 4;
5     int x = b * c;
6     int result = a + x;
7
8     printf("Result = %d", result);
9
10    return 0;
11 }
```

Output

Result = 14

=== Code Execution Successful ===

Q3 – Attribute Dependency



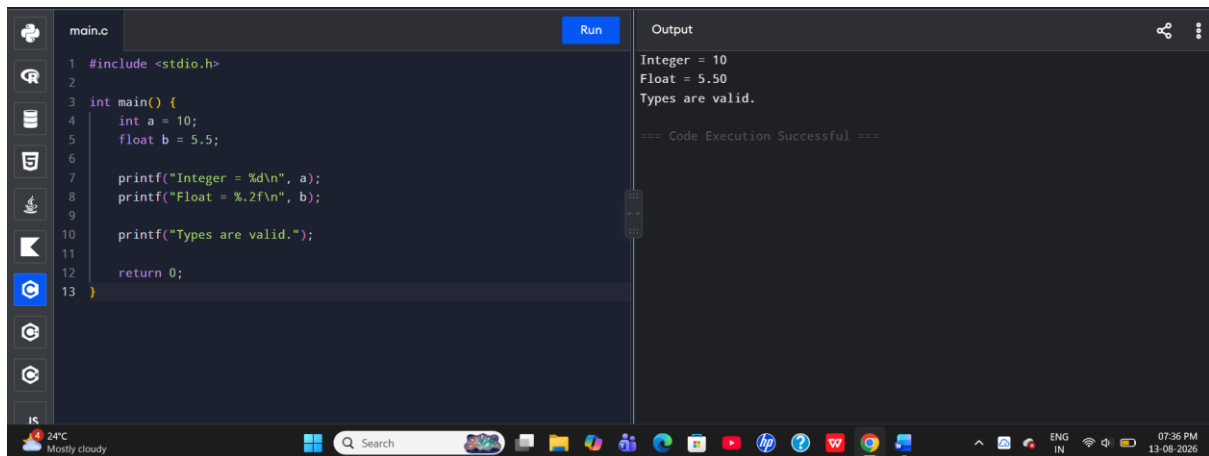
```
1 #include <stdio.h>
2
3 int main() {
4     int a = 5;
5     int b = 3;
6
7     int sum = a + b;
8     int result = sum * 2;
9
10    printf("Sum = %d\n", sum);
11    printf("Result = %d", result);
12
13    return 0;
14 }
```

Output

```
Sum = 8
Result = 16

=== Code Execution Successful ===
```

Q4 – Static Type Checking



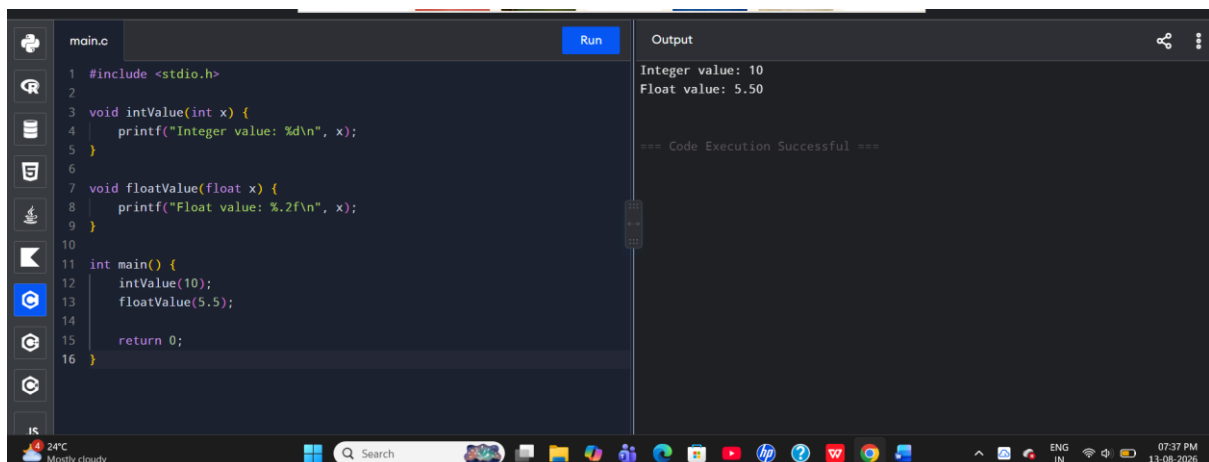
```
1 #include <stdio.h>
2
3 int main() {
4     int a = 10;
5     float b = 5.5;
6
7     printf("Integer = %d\n", a);
8     printf("Float = %.2f\n", b);
9
10    printf("Types are valid.");
11
12    return 0;
13 }
```

Output

```
Integer = 10
Float = 5.50
Types are valid.

=== Code Execution Successful ===
```

Q5 – Function Overloading / Type Resolution Demonstration



```
1 #include <stdio.h>
2
3 void intValue(int x) {
4     printf("Integer value: %d\n", x);
5 }
6
7 void floatValue(float x) {
8     printf("Float value: %.2f\n", x);
9 }
10
11 int main() {
12     intValue(10);
13     floatValue(5.5);
14
15     return 0;
16 }
```

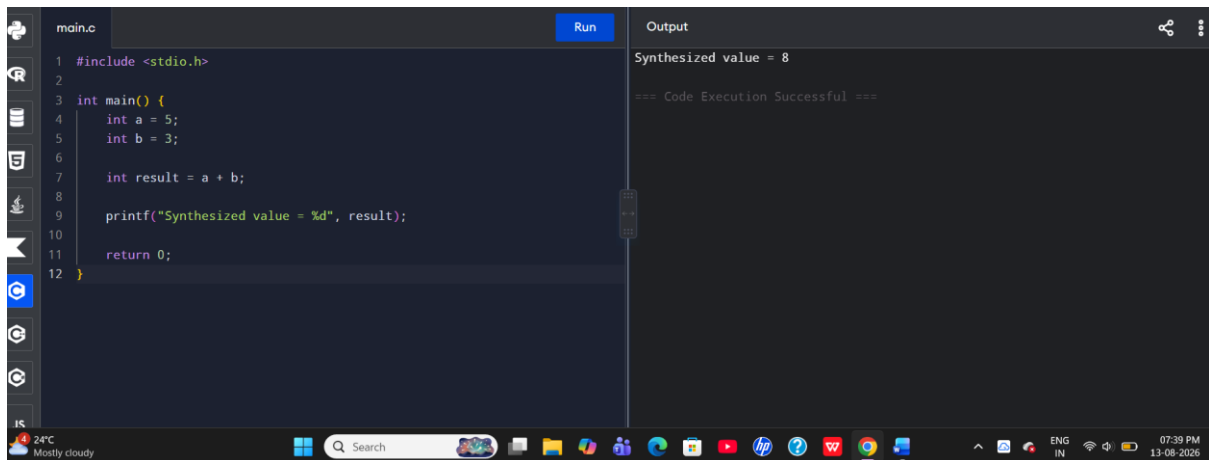
Output

```
Integer value: 10
Float value: 5.50

=== Code Execution Successful ===
```

SET 2

Q1 – Synthesized Attribute



The screenshot shows a C program in a code editor with a 'Run' button. The code defines variables a=5 and b=3, calculates result = a + b, and prints 'Synthesized value = 8'. The output window shows 'Synthesized value = 8' and '=== Code Execution Successful ==='. The Windows taskbar at the bottom shows the date and time as 07:39 PM on 13-08-2026.

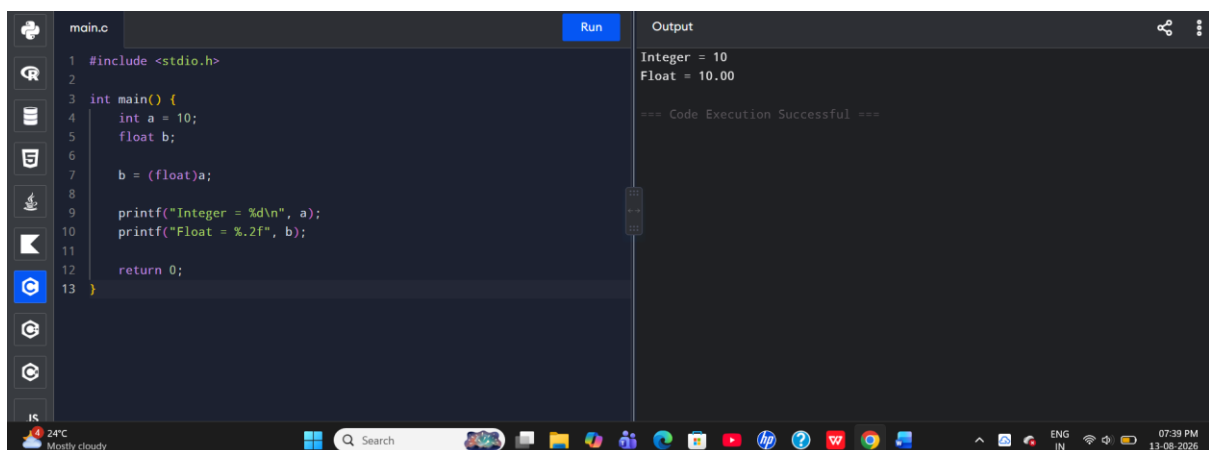
```
1 #include <stdio.h>
2
3 int main() {
4     int a = 5;
5     int b = 3;
6
7     int result = a + b;
8
9     printf("Synthesized value = %d", result);
10
11     return 0;
12 }
```

Output

Synthesized value = 8

=== Code Execution Successful ===

Q2 – Type Conversion



The screenshot shows a C program in a code editor with a 'Run' button. The code declares an integer a=10 and a float b, then casts a to float and prints both. The output window shows 'Integer = 10' and 'Float = 10.00', followed by '=== Code Execution Successful ==='. The Windows taskbar at the bottom shows the date and time as 07:39 PM on 13-08-2026.

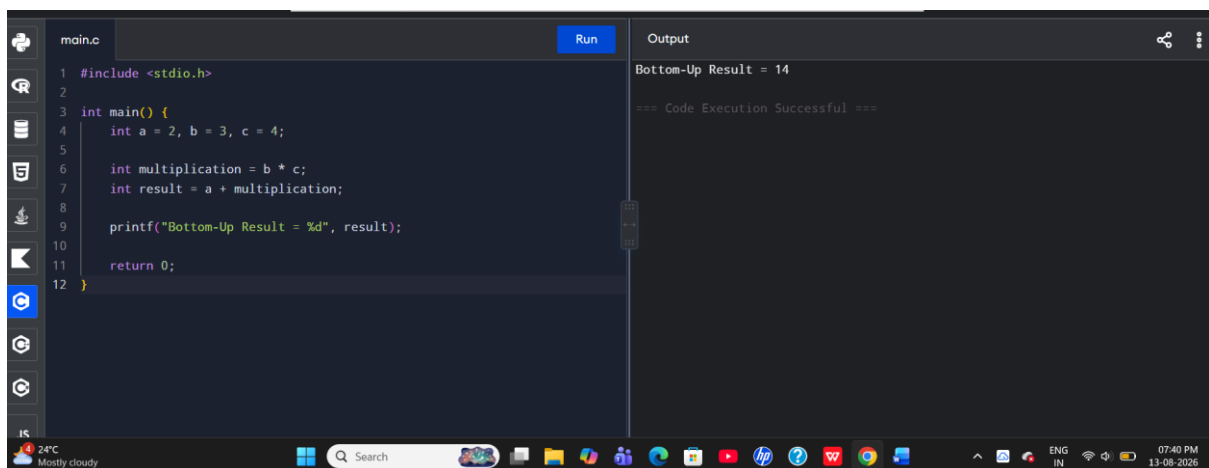
```
1 #include <stdio.h>
2
3 int main() {
4     int a = 10;
5     float b;
6
7     b = (float)a;
8
9     printf("Integer = %d\n", a);
10    printf("Float = %.2f", b);
11
12    return 0;
13 }
```

Output

Integer = 10
Float = 10.00

=== Code Execution Successful ===

Q3 – Syntax Tree and Bottom-Up Evaluation



The screenshot shows a C program in a code editor with a 'Run' button. The code initializes a=2, b=3, and c=4, calculates multiplication = b * c, then result = a + multiplication, and prints 'Bottom-Up Result = 14'. The output window shows 'Bottom-Up Result = 14' and '=== Code Execution Successful ==='. The Windows taskbar at the bottom shows the date and time as 07:40 PM on 13-08-2026.

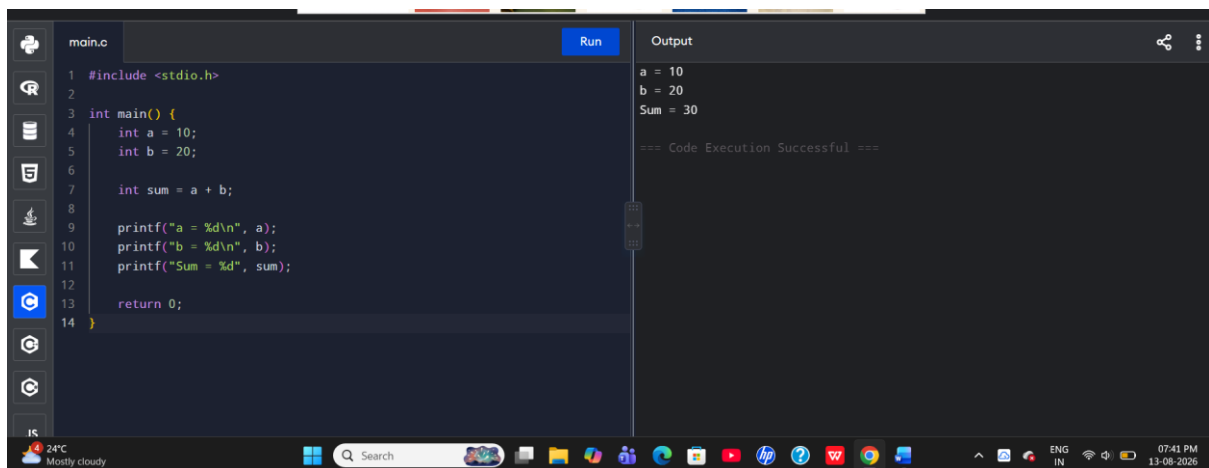
```
1 #include <stdio.h>
2
3 int main() {
4     int a = 2, b = 3, c = 4;
5
6     int multiplication = b * c;
7     int result = a + multiplication;
8
9     printf("Bottom-Up Result = %d", result);
10
11     return 0;
12 }
```

Output

Bottom-Up Result = 14

=== Code Execution Successful ===

Q4 – Left-to-Right Attribute Evaluation



The screenshot shows a C program in a code editor. The code defines two integers, `a` and `b`, with values 10 and 20 respectively. It then calculates their sum and prints the values of `a`, `b`, and the sum. The output window shows the results of the program execution.

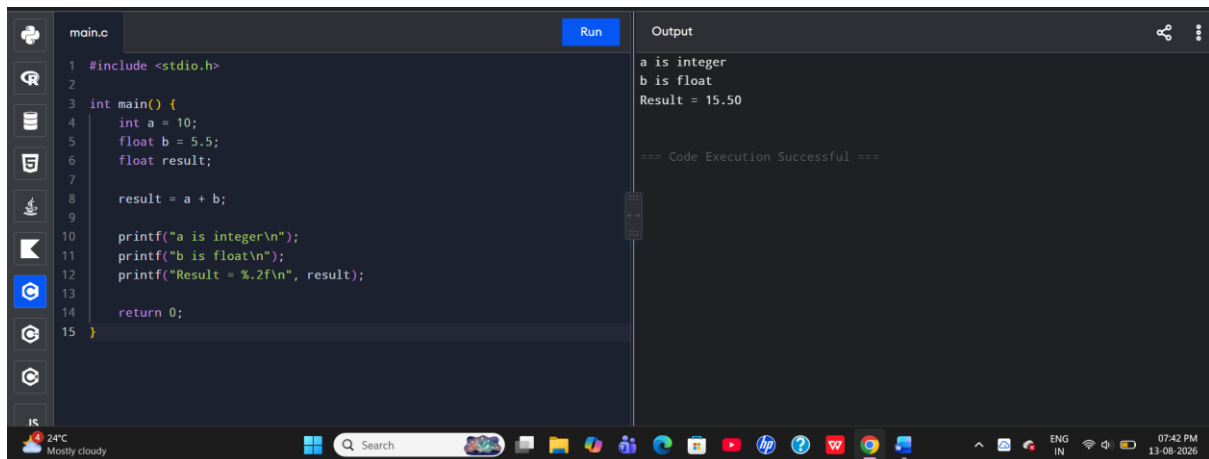
```
1 #include <stdio.h>
2
3 int main() {
4     int a = 10;
5     int b = 20;
6
7     int sum = a + b;
8
9     printf("a = %d\n", a);
10    printf("b = %d\n", b);
11    printf("Sum = %d", sum);
12
13    return 0;
14 }
```

Output:

```
a = 10
b = 20
Sum = 30

=== Code Execution Successful ===
```

Q5 – Type Checking



The screenshot shows a C program in a code editor. The code defines an integer `a` with value 10 and a float `b` with value 5.5. It then calculates the sum of `a` and `b` and prints the types of `a` and `b` along with the result. The output window shows the results of the program execution.

```
1 #include <stdio.h>
2
3 int main() {
4     int a = 10;
5     float b = 5.5;
6     float result;
7
8     result = a + b;
9
10    printf("a is integer\n");
11    printf("b is float\n");
12    printf("Result = %.2f\n", result);
13
14    return 0;
15 }
```

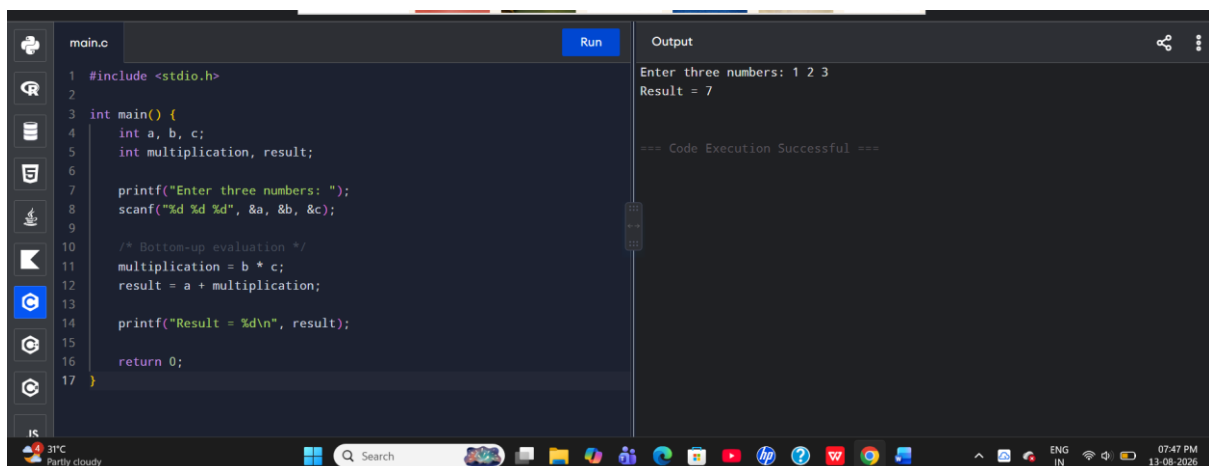
Output:

```
a is integer
b is float
Result = 15.50

=== Code Execution Successful ===
```

SET 3

Q1. Syntax-Directed Definition – Expression Evaluation



The screenshot shows a C program in a code editor. The code defines three integers, `a`, `b`, and `c`, and calculates the result of the expression `a + b * c` using bottom-up evaluation. It prints the result. The output window shows the results of the program execution.

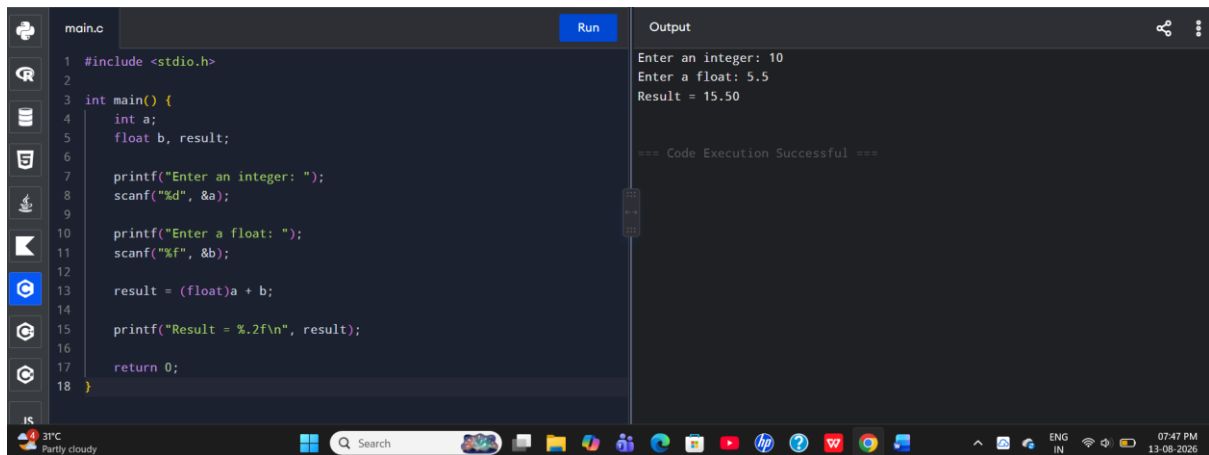
```
1 #include <stdio.h>
2
3 int main() {
4     int a, b, c;
5     int multiplication, result;
6
7     printf("Enter three numbers: ");
8     scanf("%d %d %d", &a, &b, &c);
9
10    /* Bottom-up evaluation */
11    multiplication = b * c;
12    result = a + multiplication;
13
14    printf("Result = %d\n", result);
15
16    return 0;
17 }
```

Output:

```
Enter three numbers: 1 2 3
Result = 7

=== Code Execution Successful ===
```

Q2. Type Checker with Type Conversion

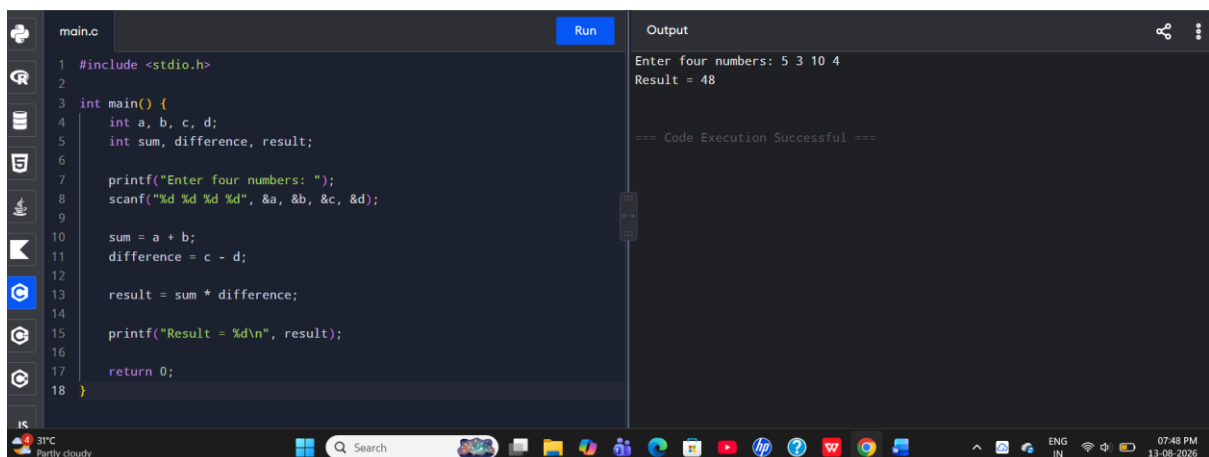


```
main.c Run Output
1 #include <stdio.h>
2
3 int main() {
4     int a;
5     float b, result;
6
7     printf("Enter an integer: ");
8     scanf("%d", &a);
9
10    printf("Enter a float: ");
11    scanf("%f", &b);
12
13    result = (float)a + b;
14
15    printf("Result = %.2f\n", result);
16
17    return 0;
18 }
```

Enter an integer: 10
Enter a float: 5.5
Result = 15.50

=== Code Execution Successful ===

Q3. Bottom-Up Expression Evaluation

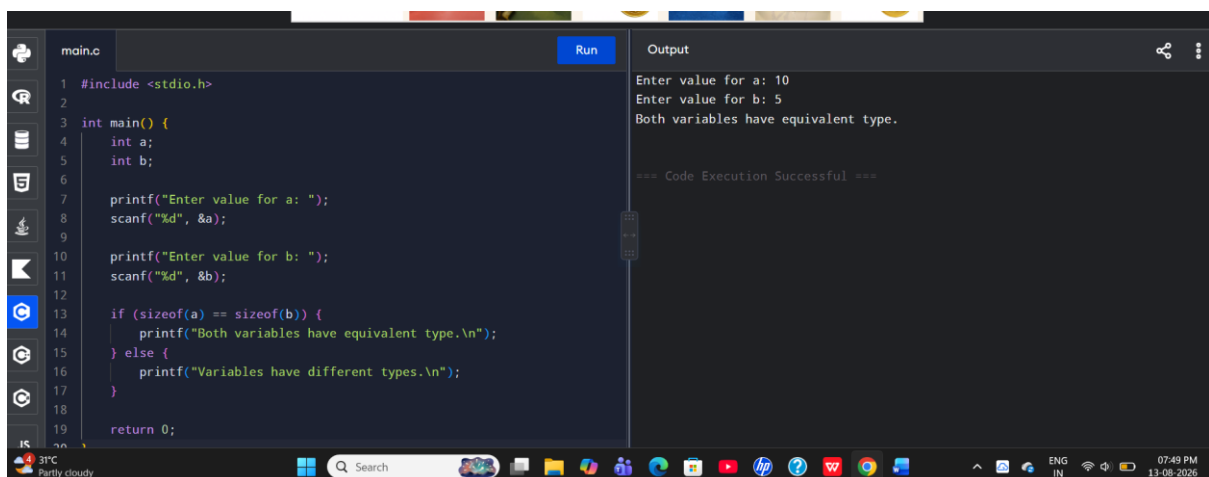


```
main.c Run Output
1 #include <stdio.h>
2
3 int main() {
4     int a, b, c, d;
5     int sum, difference, result;
6
7     printf("Enter four numbers: ");
8     scanf("%d %d %d %d", &a, &b, &c, &d);
9
10    sum = a + b;
11    difference = c - d;
12
13    result = sum * difference;
14
15    printf("Result = %d\n", result);
16
17    return 0;
18 }
```

Enter four numbers: 5 3 10 4
Result = 48

=== Code Execution Successful ===

Q4. Type Equivalence Checking

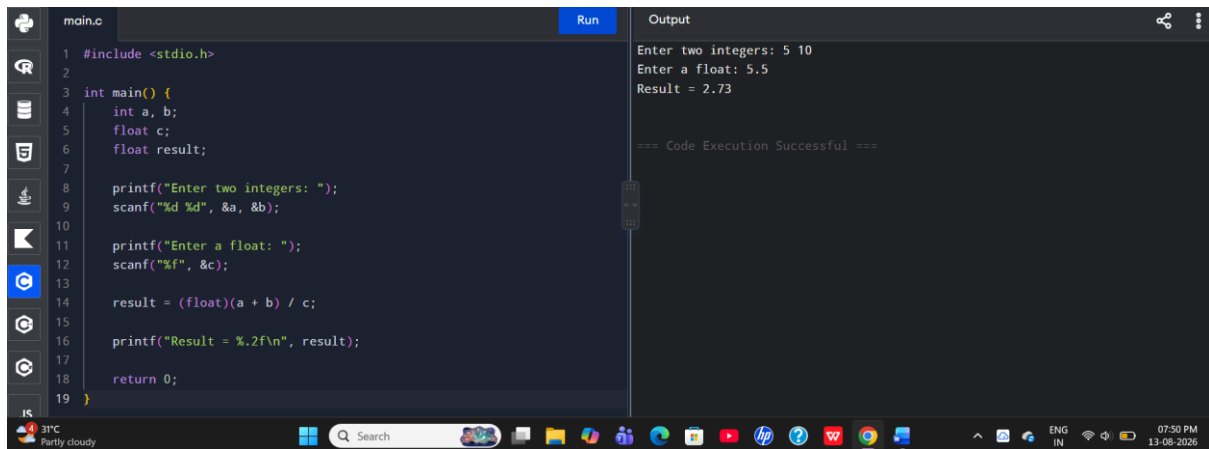


```
main.c Run Output
1 #include <stdio.h>
2
3 int main() {
4     int a;
5     int b;
6
7     printf("Enter value for a: ");
8     scanf("%d", &a);
9
10    printf("Enter value for b: ");
11    scanf("%d", &b);
12
13    if (sizeof(a) == sizeof(b)) {
14        printf("Both variables have equivalent type.\n");
15    } else {
16        printf("Variables have different types.\n");
17    }
18
19    return 0;
20 }
```

Enter value for a: 10
Enter value for b: 5
Both variables have equivalent type.

=== Code Execution Successful ===

Q5. Type Checking and Type Conversion



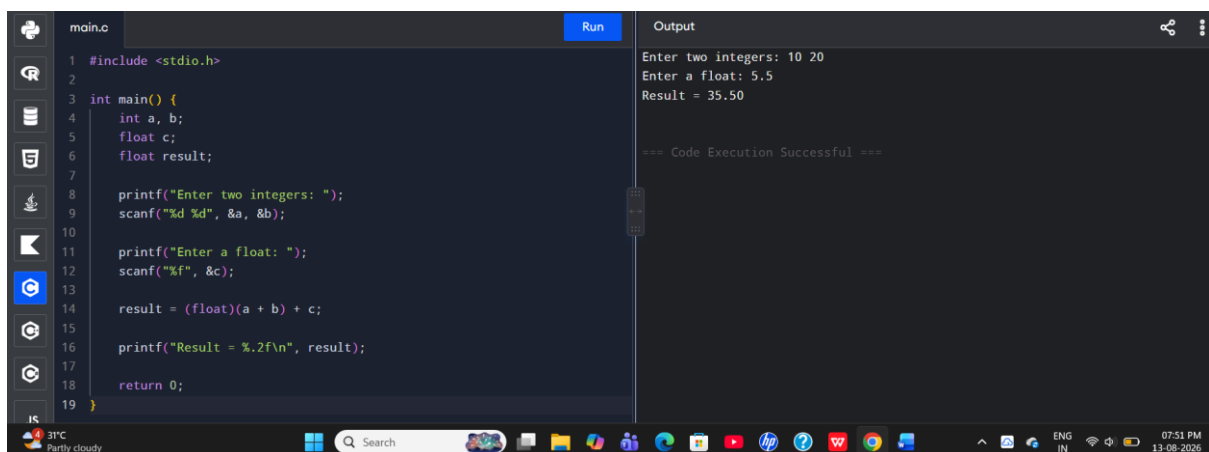
```
main.c Run Output
1 #include <stdio.h>
2
3 int main() {
4     int a, b;
5     float c;
6     float result;
7
8     printf("Enter two integers: ");
9     scanf("%d %d", &a, &b);
10
11    printf("Enter a float: ");
12    scanf("%f", &c);
13
14    result = (float)(a + b) / c;
15
16    printf("Result = %.2f\n", result);
17
18    return 0;
19 }
```

Enter two integers: 5 10
Enter a float: 5.5
Result = 2.73

=== Code Execution Successful ===

SET 4

Q1. Syntax-Directed Definition and Type Conversion

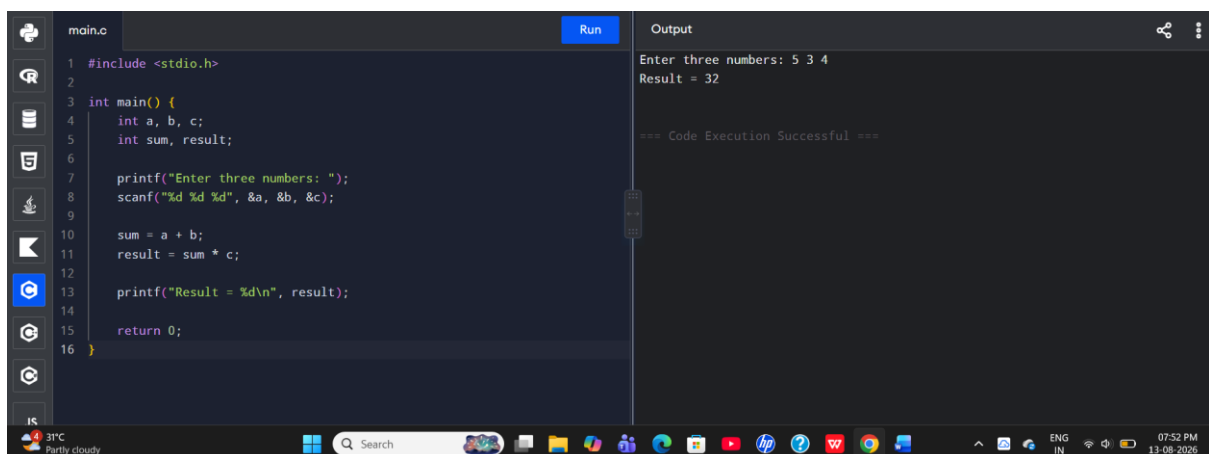


```
main.c Run Output
1 #include <stdio.h>
2
3 int main() {
4     int a, b;
5     float c;
6     float result;
7
8     printf("Enter two integers: ");
9     scanf("%d %d", &a, &b);
10
11    printf("Enter a float: ");
12    scanf("%f", &c);
13
14    result = (float)(a + b) + c;
15
16    printf("Result = %.2f\n", result);
17
18    return 0;
19 }
```

Enter two integers: 10 20
Enter a float: 5.5
Result = 35.50

=== Code Execution Successful ===

Q2. Top-Down and Bottom-Up Evaluation



```
main.c Run Output
1 #include <stdio.h>
2
3 int main() {
4     int a, b, c;
5     int sum, result;
6
7     printf("Enter three numbers: ");
8     scanf("%d %d %d", &a, &b, &c);
9
10    sum = a + b;
11    result = sum * c;
12
13    printf("Result = %d\n", result);
14
15    return 0;
16 }
```

Enter three numbers: 5 3 4
Result = 32

=== Code Execution Successful ===

Q3. Inherited and Synthesized Attributes

The screenshot shows a C program in a code editor with a 'Run' button and an 'Output' pane. The code is as follows:

```
1 #include <stdio.h>
2
3 int main() {
4     int value, multiplier;
5     int result;
6
7     printf("Enter value: ");
8     scanf("%d", &value);
9
10    printf("Enter multiplier: ");
11    scanf("%d", &multiplier);
12
13    result = value * multiplier;
14
15    printf("Result = %d\n", result);
16
17    return 0;
18 }
```

The output pane shows the following text:

```
Enter value: 10
Enter multiplier: 5
Result = 50

=== Code Execution Successful ===
```

The Windows taskbar at the bottom shows the date and time as 07:53 PM on 13-08-2026.

Q4. Type Equivalence and Type Conversion

The screenshot shows a C program in a code editor with a 'Run' button and an 'Output' pane. The code is as follows:

```
1 #include <stdio.h>
2
3 int main() {
4     int a;
5     float b;
6     float result;
7
8     printf("Enter an integer: ");
9     scanf("%d", &a);
10
11    printf("Enter a float: ");
12    scanf("%f", &b);
13
14    result = (float)a + b;
15
16    printf("Result = %.2f\n", result);
17
18    return 0;
19 }
```

The output pane shows the following text:

```
Enter an integer: 10
Enter a float: 4.5
Result = 14.50

=== Code Execution Successful ===
```

The Windows taskbar at the bottom shows the date and time as 07:53 PM on 13-08-2026.

Q5. S-Attributed and L-Attributed Definitions

The screenshot shows a C program in a code editor with a 'Run' button and an 'Output' pane. The code is as follows:

```
1 #include <stdio.h>
2
3 int main() {
4     int a, b, c;
5     int multiplication;
6     int result;
7
8     printf("Enter three numbers: ");
9     scanf("%d %d %d", &a, &b, &c);
10
11    multiplication = b * c;
12    result = a + multiplication;
13
14    printf("Result = %d\n", result);
15
16    return 0;
17 }
```

The output pane shows the following text:

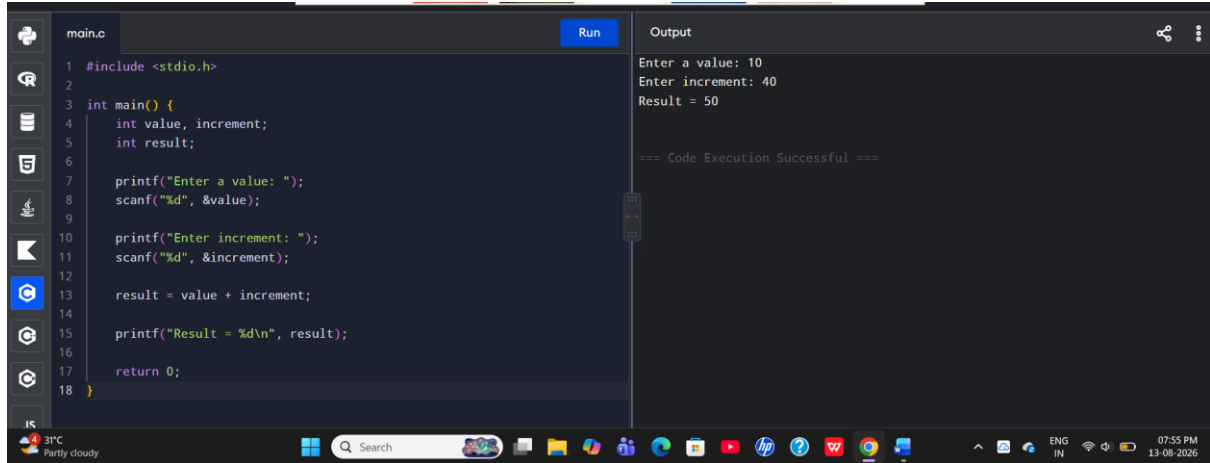
```
Enter three numbers: 2 4 5
Result = 22

=== Code Execution Successful ===
```

The Windows taskbar at the bottom shows the date and time as 07:54 PM on 13-08-2026.

SET 5

Q1. L-Attributed Definition

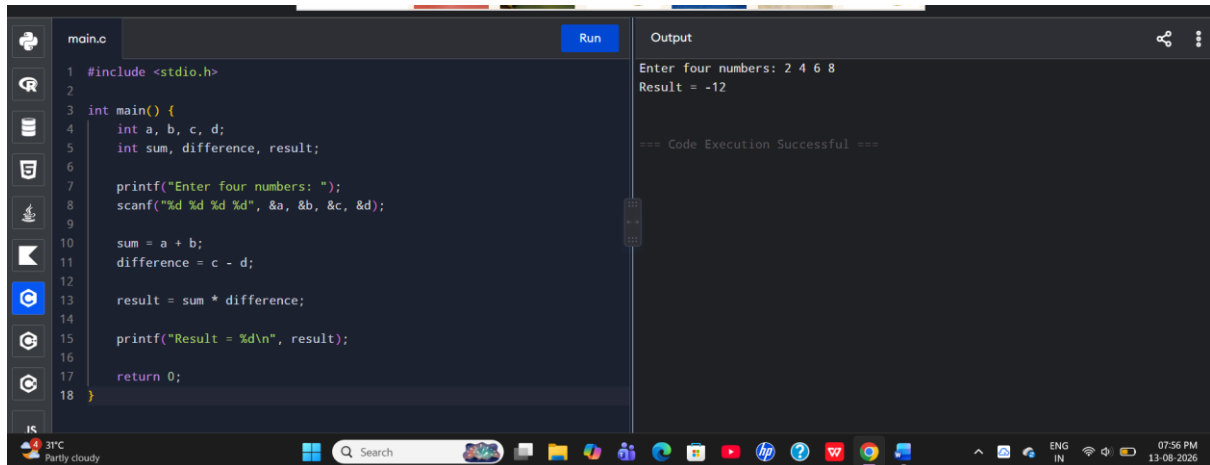


```
main.c Run Output
1 #include <stdio.h>
2
3 int main() {
4     int value, increment;
5     int result;
6
7     printf("Enter a value: ");
8     scanf("%d", &value);
9
10    printf("Enter increment: ");
11    scanf("%d", &increment);
12
13    result = value + increment;
14
15    printf("Result = %d\n", result);
16
17    return 0;
18 }
```

Enter a value: 10
Enter increment: 40
Result = 50

=== Code Execution Successful ===

Q2. Bottom-Up Evaluation



```
main.c Run Output
1 #include <stdio.h>
2
3 int main() {
4     int a, b, c, d;
5     int sum, difference, result;
6
7     printf("Enter four numbers: ");
8     scanf("%d %d %d %d", &a, &b, &c, &d);
9
10    sum = a + b;
11    difference = c - d;
12
13    result = sum * difference;
14
15    printf("Result = %d\n", result);
16
17    return 0;
18 }
```

Enter four numbers: 2 4 6 8
Result = -12

=== Code Execution Successful ===

Q3. Type Equivalence

The screenshot shows a C program in a code editor with a dark theme. The code is as follows:

```
1 #include <stdio.h>
2
3 int main() {
4     int a, b;
5
6     printf("Enter two integer values: ");
7     scanf("%d %d", &a, &b);
8
9     if (sizeof(a) == sizeof(b)) {
10        printf("The types are equivalent.\n");
11    } else {
12        printf("The types are not equivalent.\n");
13    }
14
15    return 0;
16 }
```

The output window on the right shows the execution results:

```
Enter two integer values: 5 10
The types are equivalent.

=== Code Execution Successful ===
```

The Windows taskbar at the bottom shows the date and time as 07:56 PM on 13-08-2026.

Q4. Type Checking and Type Conversion

The screenshot shows a C program in a code editor with a dark theme. The code is as follows:

```
1 #include <stdio.h>
2
3 int main() {
4     int a, b;
5     float c;
6     float result;
7
8     printf("Enter two integers: ");
9     scanf("%d %d", &a, &b);
10
11    printf("Enter a floating-point value: ");
12    scanf("%f", &c);
13
14    if (c != 0) {
15        result = (float)(a + b) / c;
16        printf("Result = %.2f\n", result);
17    } else {
18        printf("Division by zero is not allowed.\n");
19    }
20 }
```

The output window on the right shows the execution results:

```
Enter two integers: 10 20
Enter a floating-point value: 5
Result = 6.00

=== Code Execution Successful ===
```

The Windows taskbar at the bottom shows the date and time as 07:57 PM on 13-08-2026.

Q5. Attribute Dependency and Expression Evaluation

The screenshot shows a C program in a code editor with a dark theme. The code is as follows:

```
1 #include <stdio.h>
2
3 int main() {
4     int a, b, c, d;
5     int multiplication;
6     int addition;
7     int result;
8
9     printf("Enter four numbers: ");
10    scanf("%d %d %d %d", &a, &b, &c, &d);
11
12    multiplication = b * c;
13    addition = a + multiplication;
14    result = addition - d;
15
16    printf("Result = %d\n", result);
17
18    return 0;
19 }
```

The output window on the right shows the execution results:

```
Enter four numbers: 2 3 4 5
Result = 9

=== Code Execution Successful ===
```

The Windows taskbar at the bottom shows the date and time as 07:58 PM on 13-08-2026.